PART 5 — COMPLETE SPRING BOOT MASTER NOTES

Spring Boot Basic → Advanced | REST • JPA • Validation • Testing • Security • Microservices

Detailed standalone handbook: Basic → Intermediate → Advanced, with syntax, complete examples, expected output/behavior, debugging, projects, interview and viva preparation.

ROADMAP

- Spring fundamentals
- IoC/DI/Beans
- Spring Boot startup
- Configuration/profiles
- MVC/REST
- Three-layer architecture
- JPA/Hibernate
- Entities/relationships
- Repositories
- Transactions
- Validation
- Exception handling
- Testing
- Security/JWT concepts
- DTOs/pagination
- Microservices
- Gateway/config/observability concepts
- Docker/deployment concepts
- Projects
- Interview/viva

1. Spring and Spring Boot Fundamentals

Spring provides dependency injection and a broad application framework. Spring Boot simplifies configuration, dependency management and application startup.

Core concepts

• Dependency Injection: objects receive dependencies instead of constructing them directly.

• Inversion of Control: framework manages object creation/lifecycle.

• Bean: object managed by the Spring container.

• Configuration: defines how application components are assembled.

Basic application

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(
            Application.class, args);
    }
}
```

Why Spring Boot?

• Starter dependencies

• Auto-configuration

• Embedded server

• Externalized configuration

• Production-oriented features

2. Three-Tier / Three-Layer Architecture

A common Spring application separates presentation, business logic and data access.

Flow

```
HTTP Request
    ↓
Controller
    ↓
Service
    ↓
Repository
    ↓
Database

Database result
    ↑
Repository
    ↑
Service
    ↑
Controller
    ↑
HTTP Response
```

Controller

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

    private final StudentService service;

    public StudentController(StudentService service) {
```

```
        this.service = service;
    }

    @GetMapping
    public List<Student> findAll() {
        return service.findAll();
    }
}
```

Service

```
@Service
public class StudentService {

    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public List<Student> findAll() {
        return repository.findAll();
    }
}
```

Repository

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

3. REST APIs

REST APIs expose resources through HTTP methods and status codes.

CRUD mapping

HTTP	Typical use
GET	Read
POST	Create
PUT	Replace/update
PATCH	Partial update
DELETE	Delete

CRUD controller

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

    @GetMapping("/{id}")
    public Student get(@PathVariable Long id) {
        return service.findById(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Student create(@RequestBody StudentRequest req) {
        return service.create(req);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        service.delete(id);
    }
}
```

Path and query parameters

```
@GetMapping("/{id}")
Student get(@PathVariable Long id) { ... }

@GetMapping
List<Student> search(
    @RequestParam(required=false) String name
) { ... }
```

4. Domain Modeling + JPA

Spring Data JPA maps Java objects to relational database tables through entities.

Entity

```
@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(unique = true)
    private String email;

    // constructors/getters/setters
}
```

Repository

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {

    List<Student> findByNameContainingIgnoreCase(
        String name
    );

    Optional<Student> findByEmail(String email);
}
```

Relationships

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "department_id")
private Department department;
```

Key point: Entity relationships require careful fetch, cascade and serialization choices. Avoid blindly returning complex entity graphs from APIs.

5. Data Layer + Transactions

Transactions keep related data changes consistent.

Transactional service

```
@Service
public class TransferService {

    @Transactional
    public void transfer(
        Long fromId,
        Long toId,
        BigDecimal amount
    ) {
        // debit
        // credit
        // both commit or both roll back
    }
}
```

Configuration

```
# application.properties
spring.datasource.url=jdbc:postgresql://localhost:5432/college
spring.datasource.username=postgres
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
```

Profiles

```
# application-dev.properties
spring.datasource.url=...

# application-prod.properties
spring.datasource.url=...
```

6. Validation + Exception Handling

Validate input at API boundaries and return consistent error responses.

Validation

```
public record StudentRequest(
    @NotBlank String name,
    @Email @NotBlank String email
) {}
```

Controller validation

```
@PostMapping
public Student create(
    @Valid @RequestBody StudentRequest request
) {
    return service.create(request);
}
```

Global exception handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    ResponseEntity<?> notFound(
        StudentNotFoundException ex) {

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(Map.of("error", ex.getMessage()));
    }
}
```

7. Testing Spring Boot

Testing should cover business behavior, web endpoints and data access at appropriate levels.

Unit test

```
@ExtendWith(MockitoExtension.class)
class StudentServiceTest {

    @Mock StudentRepository repository;

    @InjectMocks StudentService service;

    @Test
    void findsStudents() {
        when(repository.findAll())
            .thenReturn(List.of(...));

        var result = service.findAll();

        assertEquals(1, result.size());
```

```
    }
}
```

Spring Boot test

```
@SpringBootTest
class ApplicationTest {

    @Test
    void contextLoads() {}
}
```

Mock MVC concept

```
@WebMvcTest(StudentController.class)
class StudentControllerTest {
    // mock service and test HTTP behavior
}
```

8. Security + JWT Architecture

Spring Security protects application resources. JWT is commonly used for stateless API authentication, but secure implementation requires correct key management, expiry, validation and authorization.

Conceptual flow

```
Login credentials
      ↓
Authentication endpoint
      ↓
Verify credentials
      ↓
Issue access token
      ↓
Client sends:
Authorization: Bearer <token>
      ↓
Security filter validates token
      ↓
Controller / protected resource
```

Security configuration concept

```
@Bean
SecurityFilterChain security(HttpSecurity http)
        throws Exception {

    return http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated())
        .build();
}
```

Key point: Exact security configuration depends on application architecture and Spring Security version. Never copy authentication code without understanding token storage, CSRF, CORS, password hashing and authorization.

9. Microservices

Microservices split a system into independently deployable services around business capabilities.

Architecture

```
                API Gateway
               /     |      \
              /      |       \
        User Service  Order Service  Product Service
             |            |              |
          User DB      Order DB       Product DB

               ↓
        Messaging / Events
```

```
        Observability / Config
```

Service design

• Define a clear business capability.

• Own data appropriately.

• Expose stable APIs/events.

• Handle retries/timeouts.

• Use centralized observability.

• Avoid distributed transactions where possible.

• Automate deployment and configuration.

Service endpoint

```
@RestController
@RequestMapping("/api/orders")
class OrderController {

    @PostMapping
    Order create(@RequestBody CreateOrderRequest request) {
        return service.create(request);
    }
}
```

10. Advanced Backend Topics

Production Spring applications commonly add DTOs, pagination, caching, observability, database migrations, messaging and containerization.

DTO

```
public record StudentResponse(
    Long id,
    String name,
    String email
) {}
```

Pagination

```
@GetMapping
Page<StudentResponse> findAll(
    @PageableDefault(size = 20) Pageable pageable
) {
    return service.findAll(pageable);
}
```

Actuator concept

```
# application.properties
management.endpoints.web.exposure.include=health,info
```

Docker concept

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]
```

Key point: Use database migration tooling such as Flyway or Liquibase in real projects so schema changes are versioned and repeatable.

PRACTICAL PROJECTS

Project 1 — Student Management REST API

Build Controller → Service → Repository → PostgreSQL.

```
@RestController
@RequestMapping("/api/students")
class StudentController {
    private final StudentService service;

    StudentController(StudentService service) {
        this.service = service;
    }

    @GetMapping
    List<StudentResponse> all() {
        return service.findAll();
    }

    @PostMapping
    ResponseEntity<StudentResponse> create(
        @Valid @RequestBody StudentRequest request) {

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(service.create(request));
    }
}
```

Expected result/preview: run the example in the stated environment. The visible result should match the described component or behavior. Use DevTools/console/network tools to verify it.

Project 2 — JPA Entity + Repository

Model a persistent student record.

```
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;
}
```

Expected result/preview: run the example in the stated environment. The visible result should match the described component or behavior. Use DevTools/console/network tools to verify it.

Project 3 — Microservice split

Start with a modular monolith, then extract a service around a stable business capability.

```
Client
  ↓
Gateway
  ■■■ Student Service → Student DB
  ■■■ Course Service  → Course DB
  ■■■ Auth Service    → Auth DB

Use HTTP/events between services.
Add timeouts, retries, tracing and centralized logs.
```

Expected result/preview: run the example in the stated environment. The visible result should match the described component or behavior. Use DevTools/console/network tools to verify it.

INTERVIEW + VIVA REVISION

• What is Spring?

• Spring vs Spring Boot?

• What is IoC?

• What is dependency injection?

• What is a Bean?

• What does @SpringBootApplication do?

• Controller vs Service vs Repository?

• What is REST?

• GET vs POST?

• What is @RequestBody?

• @PathVariable vs @RequestParam?

• What is JPA?

• JPA vs Hibernate?

• What is an Entity?

• @Id?

• Lazy vs eager loading?

• What is a transaction?

• @Transactional?

• What is DTO?

• Why not expose entities directly?

• What is validation?

• What is @RestControllerAdvice?

• How do you test Spring Boot?

• What is Spring Security?

• JWT flow?

• What is microservices architecture?

• Why separate databases?

• What is an API gateway?

• Why observability?

• Why migrations?

FINAL CHEAT SHEET

Revise the syntax patterns, lifecycle/flow, common errors, and project architecture. The goal is not memorizing every property or method, but knowing the model well enough to build and debug applications.